

Paso 10 - Clases con métodos static | Solucionario

Las cinco soluciones del paso 10. Cada ejercicio son dos archivos: la clase de herramientas y el programa que la usa. El B10.5 son respuestas, no código.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso10` y `javac *.java`.

B10.1 - Agrandar la clase Notas

Tres métodos más en `Notas`. `porcentajeAprobacion` no vuelve a recorrer el arreglo: usa `contarRojas`, que ya estaba.

▼ Ver la solución (Notas.java, B10_1_MasNotas.java)

Notas.java

```

/*
 * NOTAS - la del paso 10, con los tres metodos del B10.1 agregados al final.
 */
public class Notas {

    public static final double NOTA_MINIMA = 1.0;
    public static final double NOTA_MAXIMA = 7.0;
    public static final double NOTA_APROBACION = 4.0;

    public static double promedio(double[] notas) {
        double suma = 0;
        for (double n : notas) {
            suma += n;
        }
        return suma / notas.length;
    }

    public static double maxima(double[] notas) {
        double mayor = notas[0];
        for (double n : notas) {
            if (n > mayor) {
                mayor = n;
            }
        }
        return mayor;
    }

    public static int contarRojas(double[] notas) {
        int rojas = 0;
        for (double n : notas) {
            if (n < NOTA_APROBACION) {
                rojas++;
            }
        }
        return rojas;
    }

    public static boolean aprobo(double promedio) {
        return promedio >= NOTA_APROBACION;
    }

    // ----- B10.1 -----

```

```

public static double minima(double[] notas) {
    double menor = notas[0];
    for (double n : notas) {
        if (n < menor) {
            menor = n;
        }
    }
    return menor;
}

public static int contarAzules(double[] notas) {
    int azules = 0;
    for (double n : notas) {
        if (n > 5.0) {
            azules++;
        }
    }
    return azules;
}

public static double porcentajeAprobacion(double[] notas) {
    int aprobadas = notas.length - contarRojas(notas);    // usa el que ya
existe
    return aprobadas * 100.0 / notas.length;
}
}

```

B10_1_MasNotas.java

```

public class B10_1_MasNotas {
    public static void main(String[] args) {
        double[] notas = {5.5, 6.2, 3.8, 4.5};

        System.out.println("Minima: " + Notas.minima(notas));
        System.out.println("Azules: " + Notas.contarAzules(notas));
        System.out.printf("Aprobacion: %.1f%%\n",
Notas.porcentajeAprobacion(notas));
    }
}

```

Salida:

```
Minima: 3.8
Azules: 2
Aprobacion: 75.0%
```

Lo que importa. Dentro de `Notas`, los métodos se llaman **sin** prefijo: `contarRojas(notas)` y no `Notas.contarRojas(notas)`. El prefijo es solo para llamar desde **otra** clase.

El `100.0` de `porcentajeAprobacion` es la trampa del paso 3 escondida: `aprobadas * 100 / notas.length` con enteros daría 75 (aquí da lo mismo), pero con 2 de 3 daría 66 en vez de 66.7. El `.0` fuerza la división decimal.

Y `%%` en el `printf` para imprimir el signo de porcentaje, igual que en el B3.6.

B10.2 - La clase Conversor

Una clase de utilidades: sin `main`, sin estado, todo `public static`.

▼ Ver la solución (Conversor.java, B10_2_ProbarConversor.java)

Conversor.java

```
public class Conversor {

    public static final int UF = 39000;          // valor aproximado, en pesos

    public static double celsiusAFahrenheit(double c) {
        return c * 9 / 5 + 32;
    }

    public static double kmAMillas(double km) {
        return km * 0.621371;
    }

    public static double pesosAUf(int pesos) {
        return (double) pesos / UF;
    }

    public static int ufAPesos(double uf) {
        return (int) (uf * UF);
    }
}
```

B10_2_ProbarConversor.java

```
public class B10_2_ProbarConversor {
    public static void main(String[] args) {
        System.out.println("25 C son " + Conversor.celsiusAFahrenheit(25) + " F");
        System.out.printf("5 km son %.1f millas\n", Conversor.kmAMillas(5));
        System.out.printf("1500000 pesos son %.2f UF\n",
            Conversor.pesosAUf(1500000));
        System.out.println("2.5 UF son " + Conversor.ufAPesos(2.5) + " pesos");
    }
}
```

Salida:

```
25 C son 77.0 F
5 km son 3.1 millas
1500000 pesos son 38.46 UF
2.5 UF son 97500 pesos
```

`c * 9 / 5` funciona; `c * (9 / 5)` no. En el primero, `c` es `double`, así que `c * 9` ya es decimal y dividir por 5 conserva los decimales. En el segundo, los paréntesis obligan a calcular `9 / 5` entre enteros: da 1, y 25 °C te quedan en 57.0 °F. Es la trampa del paso 3 escondida en una fórmula de verdad.

`pesosAUF` necesita el `cast` `(double) pesos / UF`: los dos son `int`, así que sin él `1500000 / 39000` daría 38 pelado.

`Convertor` es lo mismo que `Math`: constantes (`UF`, como `Math.PI`) y métodos `static` (como `Math.sqrt`). Cuando entiendas que puedes escribir tu propio `Math`, entendiste el paso 10.

B10.3 - La clase Numeros

Los métodos sueltos del paso 9, ahora en su propia clase y usados desde otra.

▼ Ver la solución (Numeros.java, B10_3_ProbarNumeros.java)

Numeros.java

```
public class Numeros {

    public static boolean esPar(int n) {
        return n % 2 == 0;
    }

    public static boolean esPrimo(int n) {
        if (n < 2) {
            return false;
        }
        for (int divisor = 2; divisor < n; divisor++) {
            if (n % divisor == 0) {
                return false;
            }
        }
        return true;
    }

    public static long factorial(int n) {
        long resultado = 1;
        for (int i = 1; i <= n; i++) {
            resultado *= i;
        }
        return resultado;
    }

    public static int sumaDigitos(int n) {
        int suma = 0;
        while (n > 0) {
            suma += n % 10;
            n = n / 10;
        }
        return suma;
    }
}
```

B10_3_ProbarNumeros.java

```

public class B10_3_ProbarNumeros {
    public static void main(String[] args) {
        System.out.printf("%3s %6s %7s %12s %14s%n",
            "n", "par", "primo", "factorial", "suma digitos");

        for (int n = 1; n <= 20; n++) {
            System.out.printf("%3d %6s %7s %12d %14d%n",
                n,
                Numeros.esPar(n) ? "si" : "no",
                Numeros.esPrimo(n) ? "si" : "no",
                Numeros.factorial(n),
                Numeros.sumaDigitos(n));
        }
    }
}

```

Salida:

n	par	primo	factorial	suma digitos
1	no	no	1	1
2	si	si	2	2
3	no	si	6	3
4	si	no	24	4
5	no	si	120	5
6	si	no	720	6
7	no	si	5040	7
8	si	no	40320	8
9	no	no	362880	9
10	si	no	3628800	1
11	no	si	39916800	2
12	si	no	479001600	3
13	no	si	6227020800	4
14	si	no	87178291200	5
15	no	no	1307674368000	6
16	si	no	20922789888000	7
17	no	si	355687428096000	8
18	si	no	6402373705728000	9
19	no	si	121645100408832000	10
20	si	no	2432902008176640000	2

Lo que importa. `sumaDigitos` **modifica su propio parámetro** (`n = n / 10`). Eso es legal y no afecta en nada a quien lo llamó: el parámetro es una **copia**. Pruébalo: después de `Numeros.sumaDigitos(573)`, tu variable sigue valiendo 573.

factorial **devuelve** `long` por lo que aprendiste en el B7.2: con `int`, la tabla empezaría a mostrar números inventados desde el 13.

El `? "si" : "no"` es el operador ternario del B8.5, y aquí se gana su lugar: convertir un `boolean` en texto dentro de un `printf` sin escribir un `if` completo.

B10.4 - Desafío: la clase Texto

Cuatro métodos que trabajan carácter por carácter con `charAt` y `length()`.

▼ Ver la solución (Texto.java, B10_4_ProbarTexto.java)

Texto.java

```
public class Texto {

    public static String alReves(String texto) {
        String resultado = "";
        for (int i = texto.length() - 1; i >= 0; i--) {
            resultado += texto.charAt(i);
        }
        return resultado;
    }

    public static boolean esPalindromo(String texto) {
        String limpio = texto.toLowerCase();
        return limpio.equals(alReves(limpio));    // usa el metodo de arriba
    }

    public static int contarVocales(String texto) {
        int vocales = 0;
        String minusculas = texto.toLowerCase();
        for (int i = 0; i < minusculas.length(); i++) {
            char c = minusculas.charAt(i);
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
                vocales++;
            }
        }
        return vocales;
    }

    public static String inicialesDe(String nombreCompleto) {
        String iniciales = "";
        for (int i = 0; i < nombreCompleto.length(); i++) {
            if (i == 0 || nombreCompleto.charAt(i - 1) == ' ') {
                iniciales += nombreCompleto.charAt(i);
            }
        }
        return iniciales.toUpperCase();
    }
}
```

B10_4_ProbarTexto.java

```
public class B10_4_ProbarTexto {
    public static void main(String[] args) {
        System.out.println("alReves(\"DUOC\") -> " + Texto.alReves("DUOC"));
        System.out.println("esPalindromo(\"reconocer\") -> " +
        Texto.esPalindromo("reconocer"));
        System.out.println("esPalindromo(\"DUOC\") -> " +
        Texto.esPalindromo("DUOC"));
        System.out.println("contarVocales(\"murcielago\") -> " +
        Texto.contarVocales("murcielago"));
        System.out.println("inicialesDe(\"Ana Perez Soto\") -> " +
        Texto.inicialesDe("Ana Perez Soto"));
    }
}
```

Salida:

```
alReves("DUOC") -> COUD
esPalindromo("reconocer") -> true
esPalindromo("DUOC") -> false
contarVocales("murcielago") -> 5
inicialesDe("Ana Perez Soto") -> APS
```

Lo que importa. `esPalindromo` **usa** `alReves` en vez de escribir otro ciclo: dos métodos de la misma clase, sin prefijo. Y compara con `equals`, nunca con `==` (paso 5).

`inicialesDe` mira el carácter anterior: una letra es inicial si está al principio (`i == 0`) o si justo antes hay un espacio. Es un truco que sirve para cualquier cosa que dependa del vecino.

Un aviso sobre `resultado += texto.charAt(i)`: cada `+=` crea un `String` nuevo, porque en Java los textos son **inmutables**. Para 20 letras da lo mismo; para 100.000 es lentísimo, y ahí se usa `StringBuilder`. Es la primera vez que el curso toca ese tema, y por ahora basta con que sepas que existe.

B10.5 - Preguntas para responder por escrito

Las cuatro respuestas. Compáralas con lo que escribiste **antes** de abrir esto.

1. ¿Por qué `Notas` no tiene `main`? ¿Qué pasa con `java Notas`? Porque no es un programa: es una caja de herramientas para que la usen otros programas. `java Notas` responde `Error: Main`

`method not found in class Notas`, porque `java` busca un `main` para arrancar y ahí no hay ninguno. Una clase sin `main` se **compila** pero no se **ejecuta**.

2. ¿Por qué los métodos llevan `public`? Porque los llama **otra** clase. En el paso 9 los métodos vivían en la misma clase que el `main`, así que no hacía falta. (Dato fino: en la misma carpeta funcionaría igual sin `public`, porque comparten paquete; el `public` es lo correcto y lo que se usa siempre. Ponle `private` a uno y mira el error: `has private access in Notas`.)

3. ¿En qué se parecen `Math` y `Notas`? En todo. Las dos son clases sin `main`, con constantes (`Math.PI`, `Notas.NOTA_APROBACION`) y métodos `static` que se llaman con el nombre de la clase adelante. `Math` viene con Java y `Notas` la escribiste tú: esa es la única diferencia.

4. ¿Por qué `Scanner` necesita `new` y `Math` no? Porque un `Scanner` **recuerda cosas**: de dónde lee y por dónde va. Cada `Scanner` es un objeto distinto con su propia memoria, y por eso hay que fabricarlo con `new`. `Math` no recuerda nada: `Math.sqrt(16)` da 4 hoy, mañana y en cualquier programa. Cuando una clase solo tiene métodos que reciben datos y devuelven resultados, no necesita objetos.

Esa distinción — clases que **hacen cuentas** contra clases que **guardan estado** — es exactamente el tema de la demo 1. Un `Perro` tiene nombre, raza y edad propios: por eso se hace `new Perro(...)`.

Volver a la [hoja del paso 10](#) o al [paso 10 de la guía](#).